Praxis der Softwareentwicklung

G-Cal

Wintersemester 2025/26

Qualitätssicherung

Karlsruher Institut für Technologie (KIT)

Karlsruhe, den 13.03.2026

Karlsruher Institut für Technologie
Fakultät für Informatik
Postfach 6980
76128 Karlsruhe

Inhaltsverzeichnis

1 Einleitung **4**

2 Bugs und Verbesserungen **5**
2.1 Client 5
2.1.1 Model 5
2.1.2 ViewModel 6
2.1.3 View 6
2.2 Server 11

3 Code Qualität **14**
3.1 Statische Analyse 14
3.1.1 Server 14

4 Unit Testing und Abdeckungen **15**
4.1 Client 15
4.1.1 Model 15
4.1.2 ViewModel 16
4.1.3 View 16
4.2 Server 18

5 Integrationstests **20**

6 Systemtests **21**
6.1 Server Verfügbarkeit 21
6.2 Manuelle Tests 21
6.2.1 Login und Logout 21
6.2.2 Events 21
6.2.3 Freunde 22
6.2.4 Dark-Mode und Sprache 22
6.3 UI-Stresstests (Monkey Testing) 22

Abbildungsverzeichnis

1 Testabdeckung des Client-Moduls 18
2 Testabdeckung des Server-Moduls 19
3 Ergebnis des Basis-Stabilitätstests (100 Events) 23
4 Ergebnis der realistischen Benutzersimulation (1000 Events mit 200ms Delay) 23
5 Ergebnis der maximalen Kapazitätsprüfung (1000 Events ohne Delay) 24

1 Einleitung

Im Anschluss an den Implementierungsbericht gibt es im Folgenden eine Übersicht über die Qualitätssicherungsphase. Dabei wird auf die aufgetretenen Bugs und ihre Behebung eingegangen, auf die verschiedenen Tests, die zur Sicherstellung der Qualität von *g-cal* genutzt wurden sowie auf die Abdeckung der Funktionalität durch Unit-Tests.

2 Bugs und Verbesserungen

Im Folgenden werden alle während der Implementierungsphase und Qualitätssicherungsphase aufgetretenen Bugs erläutert. Dazu werden der Bug, der Grund für das Auftreten sowie der Fix zum Beheben des Bugs beschrieben.

2.1 Client

2.1.1 Model

Bug Fehlende Achievements in der lokalen Datenbank.

Grund Die Initialisierung der Achievements wurde beim Erstellen der Datenbank instanziiert, jedoch nicht mit Standardwerten befüllt.

Fix Bei der Instanziierung der Datenbank wird ein `ACHIEVEMENT_CALLBACK` registriert, welcher über die `onCreate`-Methode alle Achievements im Initialzustand in die Datenbank einfügt.

Bug Nach dem Login mit einem bestehenden Account erfolgt keine Synchronisation der Serverdaten mit dem Client.

Grund Die verwendeten `refresh`-Methoden basierten auf `Update`-Operationen in der lokalen Datenbank. Da die Daten nach einem Neu-Login lokal noch nicht existierten, blieben diese Operationen wirkungslos.

Fix In der `ModelFacade` wurde eine Initialisierungsmethode implementiert, die den aktuellen Stand vom Server abruft und die Daten mittels `Insert`-Operationen (bzw. `Upsert`) einmalig in die lokale Datenbank einpflegt.

Bug Fehlerhafte Abfragen bei abgeschlossenen Achievements.

Grund Die `CompletedAchievement`-Queries geben statische Listen statt eines `Flows` zurück, wodurch die Ausführung blockierend auf dem Main-Thread erfolgt.

Fix Deklaration der betroffenen Funktionen als `suspend`, um die Datenbankoperationen vom Main-Thread zu entkoppeln.

Bug Shared Events konnten nicht gelöscht werden

Grund Die Methode zum löschen von Events in ModelData beinhaltete keinen Fall für Shared Events und auch im EventRepo hat keine Methode zum löschen von Shared Events existiert.

Fix Die Methoden wurden entsprechend angepasst/hinzugefügt

2.1.2 ViewModel

Bug Gamification-Level startete initial bei 0 statt bei 1.

Grund Die Level-Berechnung erfolgte im `ProfileViewModel` durch eine einfache Integer-Division der Gesamt-XP (`totalXp / 100`). Ein neuer Nutzer mit 0 XP wurde dadurch als Level 0“ angezeigt, was dem Gamification-Entwurf widersprach und die Fortschrittsanzeige (Progress Bar) inkonsistent machte.

Fix Die Berechnungslogik in `updateGamificationStats` wurde auf ein 1-basiertes System umgestellt (`(totalXp / 100) + 1`). Zusätzlich wurde die Formel für `xpForNextLevel` angepasst, um den absoluten Schwellenwert korrekt auf das neue Basis-Level zu referenzieren, wodurch eine lückenlose Synchronisation zwischen XP-Stand und UI-Balken erreicht wurde.

2.1.3 View

Bug Sprachwechsel funktionierte nicht korrekt.

Grund Die Spracheinstellung wurde nur im transienten UI-State gespeichert, aber nicht über `SharedPreferences` persistiert. Nach einem Activity-Neustart ging die Einstellung verloren.

Fix Die Spracheinstellung wird nun direkt über `AppCompatDelegate` persistiert und beim App-Start aus den `SharedPreferences` wiederhergestellt. Der `SettingsScreenView` synchronisiert den ViewModel-State mit den aktuellen Geräteeinstellungen via `LaunchedEffect`.

Bug Darkmode-Sprachwechsel-Switch funktionierte nicht korrekt.

Grund Der Sprachwechsel löst eine Activity-Recreation aus. Dabei wurde der Dark-Mode-Zustand nur im `PGrofileUiState` gehalten, aber nicht in den `SharedPreferences` gespeichert. Nach der Recreation las die App den Standardwert `false` aus.

Fix Der Dark-Mode-Zustand wird nun beim Umschalten direkt in die `SharedPreferences` geschrieben. Beim App-Start liest die `GCalApp` den persistierten Wert aus und übergibt ihn an den `AppSettingsController`.

Bug Events mit Gruppe waren im Client nicht sichtbar.

Grund Es lag eine doppelte Filterung vor: Das ViewModel filterte Events anhand der `hiddenGroupIds`, und die View-Schicht filterte zusätzlich in `getFilteredEvents()` mit einer Whitelist-Logik (`groupId in visibleGroupIds`). Neu erstellte Gruppen waren in der `visibleGroupIds`-Liste noch nicht enthalten.

Fix Die View-Schicht-Filterung wurde von einer Whitelist-Logik auf eine Blacklist-Logik umgestellt: `event.groupId == null || event.groupId !in hiddenGroupIds`. Damit werden Events nur ausgeblendet, wenn ihre Gruppe explizit vom Benutzer deaktiviert wurde.

Bug Termine ohne Gruppe waren im Client nicht sichtbar.

Grund Die `getFilteredEvents()`-Funktion in `MainScreenView` prüfte `event.groupId in visibleGroupIds`. Ein Termin ohne Gruppe erhielt die `groupId = 0L` (von `NoGroup`). Da `0L` nicht in der `visibleGroupIds`-Menge enthalten war, wurde das Event herausgefiltert.

Fix Die Umstellung auf die Blacklist-Filterlogik (siehe oben) löst dieses Problem, da `groupId = 0L` standardmäßig nicht in `hiddenGroupIds` enthalten ist und somit immer angezeigt wird.

Bug „None"-Option im Gruppen-Picker verursachte unsichtbare Events.

Grund Das Model lieferte ein `NoGroup`-Objekt mit dem Namen ,,None" über den `getAllGroups()`-Flow. Dieses wurde als reguläre Gruppe im Picker angezeigt, führte aber bei Auswahl zu einer fehlerhaften Verarbeitung in der Model-Schicht.

Fix Der `NoGroup`-Eintrag wird nun in `observeGroups()` im ViewModel mittels `.filter` herausgefiltert. Zusätzlich wurden die hardcodierten RadioButton-Einträge für ,,None" aus den `GroupPickerDialog`-Composables in `CreateEntrySheet.kt` und `EventEditorDialog.kt` entfernt.

Bug Template-Wechsel beim Bearbeiten von Events war möglich.

Grund Beim Bearbeiten eines bestehenden Events konnte der Benutzer den Event-Typ (ToDo $\leftrightarrow$ Termin $\leftrightarrow$ Shared Event) über FilterChips ändern. Dies führte zu Inkonsistenzen, da die Model-Schicht keinen Typwechsel für bestehende Entitäten unterstützt.

Fix Im `EventEditorDialog` wurden die `FilterChips` mit `enabled = false` und einem leeren `onClick`-Lambda versehen. Die Chips visualisieren weiterhin den aktuellen Typ, sind aber nicht mehr interaktiv.

Bug Hinzugefügte Freunde waren im Profil-Dialog nicht sichtbar.

Grund Beim Öffnen des `FriendsMenuDialog` fehlte die Datenbindung an den aktuellen UI-State. Obwohl die Freundesliste im `ViewModel` korrekt geladen und im `uiState` gespeichert wurde, wurde diese Information nicht an die Dialog-Komponente weitergereicht .

Fix Im `ProfileScreen` wurde der Aufruf von `FriendsMenuDialog` korrigiert, indem der Parameter `friends` explizit mit `uiState.friends` verknüpft wurde.

Bug Der angegebene Anzeigename wurde nicht gespeichert.

Grund beim insertieren des lokalen Accounts wurde die falsche Methode aufgerufen, die einen existierenden Account aktualisiert hat, aber keinen neuen erstellt hat.

Fix die richtige Methode wurde verwendet.

Bug Nach einer neuen Anmeldung oder Registrierung kam ein Whitescreen

Grund Die Aktivität LoginActivity wurde nur mit finish() beendet, wenn bereits ein Nutzer angemeldet war, nicht bei einer neuen Anmeldung oder Registrierung. Der Nutzer wurde zwar angemeldet, kam aber nicht aus dem Login-Screen raus.

Fix finish() wurde in die LoginActivity eingefügt, sobald der Nutzer angemeldet ist.

Verbesserungen

Verbesserung 1 Erweiterung des Berichte-Dialogs um ein visuelles Erfolgssystem .

Grund Zur Steigerung der Benutzermotivation sollten besondere Leistungen visuell belohnt werden. Da das Backend bereits Logik für zeitbasierte Erfolge besaß, fehlte eine entsprechende Darstellung in der Benutzeroberfläche.

Implementierung Im `ReportResultDialog` wurde eine `LazyRow` integriert, die verdiente Abzeichen wie "Früher Vogel" und "Nachteule" anzeigt.

Verbesserung 2 Entfernung der Standort- und Wiederholungsfunktionen aus der Event-Verwaltung.

Grund Sowohl das Standort-Eingabefeld als auch die Wiederholungsoptionen waren in der Model-Schicht nicht implementiert und wurden vom Backend nicht unterstützt. Die funktionslosen UI-Elemente erzeugten unnötige visuelle Komplexität.

Fix Die entsprechenden Felder und Chips wurden konsistent aus `CreateEntrySheet` und `EventEditorDialog` entfernt.

Verbesserung 3 Entfernung von Push-Benachrichtigungen und Kalender-Import.

Grund Da diese Features nicht in der Model-Schicht umgesetzt wurden, wurden die entsprechenden Steuerelemente entfernt, um eine konsistente Benutzererfahrung ohne funktionslose Schnittstellen sicherzustellen.

Fix Die zugehörigen UI-Buttons wurden aus dem Bereich der Profil-Settings entfernt.

Verbesserung 4 Gruppen-Option für ToDos entfernt.

Grund ToDos sind im System olarak persönliche Aufgaben ohne Gruppenzugehörigkeit definiert. Eine Gruppenzuweisung ist nur für Termine und Shared Events relevant, da diese im Kalender nach Gruppen gefiltert werden können.

Fix Das Gruppen-Auswahlfeld wird nun für den Event-Typ ToDo in `CreateEntrySheet` und `EventEditorDialog` explizit ausgeblendet.

2.2 Server

Bug Serialisierungsfehler zwischen Client und Server.

Grund Aufgrund von teilweise inkompatiblen Datentypen, die aufgrund von sealed interfaces nicht vernünftig serialisiert werden können, können DTOs teilweise nicht empfangen und versendet werden.

Fix Im Model des Clients wurde ein Mapping eingeführt, das beim Versenden die Client-internen Datentypen auf die Server Datentypen mappt und beim Empfangen umgekehrt.

Bug Fehler bei der Serialisierung wurden nicht gecatched.

Grund Es wurde keine Fehlerbehandlung für die (De-) Serialisierung implementiert, sodass keine genaue Fehlermeldung zurückgegeben wurde.

Fix Explizites fangen der Exceptions und loggen.

Bug Nullable Deadline bei ToDo.

Grund ToDos sollten nach Pflichtenheft eine optionale Deadline haben.

Fix Deadline Feld von ToDos wurde nullable gemacht.

Bug Server Blocking laufen lassen.

Grund Beim Testen war es nicht möglich, den Server des *RequestController* nicht-blockend laufen zu lassen, was das Testen von mehreren Unit-Tests verhinderte.

Fix Zusätzliches Blocking-Attribut, was normalerweise true ist, zum Testen aber auf false gesetzt werden kann.

Bug Log Nachrichten zu ungenau.

Grund Die Log Nachrichten waren zu ungenau/vage, was das Beheben von Bugs erschwert hat.

Fix Detailliertes Logging eingeführt.

Bug Token Decoding Fehlerbehandlung.

Grund Wenn das JWT Token decodiert wird, kann es Fehler werfen, wenn bestimmte Felder nicht gesetzt sind. Dabei wurde bisher auch nicht geprüft, ob Felder überhaupt vorhanden sind.

Fix Explizite Behandlung dieser Fehler, Prüfung ob alle benötigten Felder vorhanden sind.

Bug Delete User gibt Fehler.

Grund Aufgrund von Foreign Key Constraints gab das Löschen aller Daten eines Nutzers einen Fehler.

Fix Im Datenbankschema wurde "ON DELETE CASCADE" für die Fremdschlüssel eingeführt.

Bug Verschiedene Felder mutable.

Grund Einige Felder waren noch mutable, obwohl diese immutable sein sollten. Dazu zählen: *value* von *ExperiencePoints*, *groupID* von *Group*.

Fix Felder wurden immutable gemacht.

Bug Keine privaten Konstruktoren.

Grund DatabaseConnect und Config waren im Entwurf als Singleton geplant, allerdings wurden die Konstruktoren nicht privat gemacht.

Fix Beide Konstruktoren sind nun privat.

Bug Fehlschlag der Erstellung von Global Challenges.

Grund Schlägt die Erstellung einer Global Challenge fehl (bpsw. aufgrund einer invaliden Datenbank), so resultiert dies in einer Dauerschleife, anstatt die definierte Periode abzuwarten.

Fix Die Coroutine wartet sowohl im Erfolgsfall als auch bei einem Fehler die Periode ab.

Bug Erstellung von Shared Events übernimmt Gruppe.

Grund Teilt ein Nutzer ein Shared Event mit einer Gruppe mit anderen Nutzern, so wird diese Gruppe für alle ebenfalls übernommen, auch wenn diese überhaupt nicht für die anderen Nutzer existiert, was zu einem Fehler führt.

Fix Die Gruppe wird auf NoGroup gesetzt, kann dann von den Nutzern geupdated werden.

3 Code Qualität

3.1 Statische Analyse

Für das Testen der Code-Qualität wurde das Analyse-Tool *detekt*[1] genutzt. Im Folgenden werden die am häufigsten aufgetretenen Fehler in den beiden Komponenten erläutert.

3.1.1 Server

Zunächst gab es einige Fehlermeldungen über zu lange Zeilen und Wildcard-Imports, welche größtenteils angepasst wurden. Die Magic-Numbers im *ChallengeManager* kommen von vordefinierten Challenges, die Daten enthalten. Hier Konstanten einzuführen würde unserer Meinung nach keinen Sinn ergeben.

Außerdem wurde die lange Methode *configureRouting* im *RequestController* sowie die Anzahl der Methoden im *RequestDispatcher* bemängelt. Da dies aber dem vorgesehenen Entwurf des Servers entspricht, wurde daran nichts angepasst. Minimale Abweichungen an der Standardkonfiguration von *detekt* können unter *config/server.yml* gefunden werden.

[1] https://github.com/detekt/detekt

4 Unit Testing und Abdeckungen

4.1 Client

4.1.1 Model

Die Validierung der Software erfolgt über eine mehrstufige Teststrategie, die insbesondere die Integrität der Daten und die Korrektheit der Geschäftslogik sicherstellt.

Schnittstellentests (ClientAPI): Die Kommunikation zwischen Client und Server wird für jeden Endpunkt der `ClientAPI` durch Unit-Tests validiert. Dies stellt sicher, dass der Datenaustausch protokollkonform und robust gegenüber Fehlkonfigurationen erfolgt.

Repository-Ebene: Für das `User-` und `Event-Repository` wurden für alle öffentlichen Methoden Testfälle implementiert. Um eine isolierte Testumgebung zu schaffen, werden sowohl die lokale `Room`-Datenbank als auch die `ClientAPI` mittels Mocks simuliert.

> *Hinweis:* Nicht jeder Randfall wurde explizit durch einen Testzweig abgedeckt; stattdessen wurden seltene Pfade mit detaillierten Logs versehen, um die Rückverfolgbarkeit bei Mapping-Fehlern während der Debugging-Phase zu gewährleisten.

Logik-Validierung (XP & Achievements): Ein dedizierter Test prüft die Funktionalität des `Xp-Distributors`. Dabei wird die automatisierte Verteilung von Erfahrungspunkten an Termine und *Shared Events* sowie die korrekte Vergabe von Belohnungen verifiziert. Zusätzlich existiert für jedes `Achievement` ein Unit-Test, der die Erfüllung der Abschlusskriterien gegen die definierten Anforderungen prüft.

Lokale Datenbank Die Validierung der Persistenzschicht erfolgt über Instrumentation-Tests auf einer `In-Memory`-Datenbank. Dabei werden die Integrität der `Room`-Entitäten, sämtliche CRUD-Operationen sowie die zeitbasierte Filterung von Ereignissen verifiziert. Zusätzlich wird sichergestellt, dass Datenänderungen konsistent über `Kotlin Flows` an die Anwendung propagiert werden.

4.1.2 ViewModel

Die Geschäftslogik der ViewModels wird mit einer automatisierten Test-Suite überprüft, die auf **JUnit 4** und **MockK** basiert. Im Fokus steht dabei die korrekte Emission der `StateFlow`-Zustände in Reaktion auf Benutzeraktionen und Änderungen der angebundenen Datenquellen.

- **Test-Metriken:** Für alle ViewModels (`Main`, `Profile`, `Leaderboard`, `Report`, `Login`) wurde eine **Line-Coverage von** $> 85\,\%$ erreicht. Die Testfälle prüfen neben den *Happy Paths* vor allem die Behandlung von `IOExceptions` der Datenquelle sowie Grenzfälle bei der XP- und Level-Berechnung.
- **Asynchronität & Reaktivität:** Coroutine-abhängige Logik wird mit `kotlinx-coroutines-test` und `runTest` ausgeführt. Die emittierten `UiState`-Objekte werden mit **Turbine** ausgewertet, sodass Zustandswechsel (z. B. *Loading* $\rightarrow$ *Success*) reproduzierbar und in der erwarteten Reihenfolge auftreten.
- **Test-Isolation:** Externe Abhängigkeiten, insbesondere die `ModelFacade`, werden über *Relaxed Mocks* oder gezieltes *Stubbing* ersetzt.
- **Neustart & Ressourcen:** Zusätzlich wird das Job-Management in den ViewModels getestet. Dabei wird geprüft, dass laufende `Flow`-Subscriptions beim Abbruch des `viewModelScope` (z. B. bei Navigation) beendet werden, um Memory Leaks und Race Conditions zu vermeiden.

4.1.3 View

Automatisierte Android UI-Tests mit Jetpack Compose Um die Zuverlässigkeit, Benutzerfreundlichkeit und Wartbarkeit der Anwendung sicherzustellen, wurde eine umfassende Suite automatisierter UI-Tests auf Basis von Jetpack Compose implementiert. Der Fokus lag dabei auf der isolierten Überprüfung einzelner UI-Komponenten durch einen zustandsbasierten Testing-Ansatz.

Anstatt die Benutzeroberfläche direkt an die Business-Logik der ViewModels zu koppeln, wurden die Compose-Komponenten so gestaltet, dass sie ausschließlich UiStates entgegennehmen. Dadurch konnten die Komponenten unabhängig vom Backend oder anderen Systemteilen getestet werden. Dieser Ansatz ermöglicht schnelle, reproduzierbare und stabile UI-Tests.

Die Qualitätssicherung der Benutzeroberfläche umfasste folgende zentrale Bereiche:

Isoliertes Komponenten-Testing: UI-Komponenten wurden durch direkte Übergabe definierter UI-Zustände getestet. Dadurch konnte überprüft werden, ob unterschiedliche Zustände korrekt dargestellt werden, beispielsweise Ladeindikatoren , Fehlermeldungen oder befüllte Listenansichten.

Dynamische Formularvalidierung: Besonderes Augenmerk lag auf der Validierung von Eingabeformularen. Tests wie der `CreateEntrySheetFormTest` stellen sicher, dass der „Speichern"-Button erst aktiviert wird, wenn alle erforderlichen Felder korrekt ausgefüllt sind. Zusätzlich wurde überprüft, dass sich das UI dynamisch anpasst, wenn Benutzer zwischen verschiedenen Event-Typen wechseln.

Navigation und Backstack-Management: Die Navigation innerhalb der Anwendung wurde mithilfe eines `TestNavHost` isoliert getestet. Im `NavigationIntegrationTest` wurde insbesondere sichergestellt, dass nach einem erfolgreichen Login die `LOGIN`-Route aus dem Backstack entfernt wird.

Interaktions- und Callback-Tests: In mehreren zentralen Screens wurden Benutzerinteraktionen automatisiert überprüft. Mithilfe von Semantics-Matchern wurde getestet, ob Klicks auf Buttons, das Wechseln von Tabs oder das Aktivieren des Dark-Modes korrekt verarbeitet und an die entsprechende Logik weitergeleitet werden.

Tests wiederverwendbarer UI-Komponenten: Globale UI-Komponenten wie die `BottomNavigationBar` oder die `EventCard` wurden separat getestet, da Fehler in diesen Basiskomponenten den gesamten Navigationsfluss der Anwendung beeinträchtigen könnten.

Fazit der UI-Testphase: Durch den zustandsbasierten Testansatz konnte sichergestellt werden, dass zentrale Benutzerflüsse wie Event-Erstellung, Navigation und Authentifizierung zuverlässig funktionieren.

Current scope: all classes

Overall Coverage Summary

Package	Class, %	Method, %	Branch, %	Line, %
all classes	67.2% (202/302)	55.4% (402/725)	43.2% (395/914)	53.4% (1818/3402)

Coverage Breakdown

Package	Class, %	Method, %	Branch, %	Line, %
com.gcal.app.model.localData	4.3% (3/69)	1.2% (3/241)	0.4% (1/254)	0.2% (3/1234)
com.gcal.app.model.localData.entity	90% (9/10)	90% (9/10)		96% (48/50)
com.gcal.app.model.localData.relation	100% (3/3)	100% (3/3)		100% (11/11)
com.gcal.app.model.modelData	28.6% (2/7)	26.3% (10/38)	35.7% (35/98)	45.6% (114/250)
com.gcal.app.model.modelData.data	0% (0/1)	0% (0/3)		0% (0/5)
com.gcal.app.model.modelData.data.achievements	80% (4/5)	76.9% (10/13)	50% (3/6)	87.5% (21/24)
com.gcal.app.model.modelData.data.dto	100% (23/23)	96.9% (31/32)	55% (11/20)	98.2% (108/110)
com.gcal.app.model.modelData.data.system	69.4% (25/36)	85.3% (81/95)	44% (22/50)	90.3% (195/216)
com.gcal.app.model.modelData.repo	88.2% (15/17)	84.1% (90/107)	46.5% (92/198)	82.6% (475/575)
com.gcal.app.model.modelFacade	100% (3/3)	100% (3/3)		100% (3/3)
com.gcal.app.ui.view_model	0% (0/1)	0% (0/2)	0% (0/10)	0% (0/14)
com.gcal.app.ui.view_model.leaderboardViewModel	80% (8/10)	84.6% (11/13)	71.4% (10/14)	94.1% (64/68)
com.gcal.app.ui.view_model.loginViewModel	88.2% (15/17)	83.3% (25/30)	62.5% (25/40)	89.7% (105/117)
com.gcal.app.ui.view_model.mainViewModel	97.8% (45/46)	98.5% (66/67)	87.4% (111/127)	93.6% (337/360)
com.gcal.app.ui.view_model.profileViewModel	92.1% (35/38)	89.4% (42/47)	96.3% (52/54)	89.4% (203/227)
com.gcal.app.ui.view_model.reportViewModel	81.2% (13/16)	85.7% (18/21)	76.7% (33/43)	94.9% (131/138)

generated on 2026-03-13 19:17

Abbildung 1: Testabdeckung des Client-Moduls

4.2 Server

Da für den Server aus der Implementierungsphase kaum Unit-Test bestanden, wurden diese von Grund auf entwickelt. Die Tests wurden mit *JUnit 5*[2] und *Mockk*[3] entwickelt. Für das Testen der Umgebungsvariablen, die in der *Config* eingelesen werden, wurde die *SystemStubsExtension*[4] verwendet. Diese erlaubt das Mocken von Umgebungsvariablen für einzelne Unit-Tests, was sonst in Kotlin nicht möglich ist.

Da das Mocken von Ktorm im Allgemeinen eher schwierig ist, haben wir uns dazu entschieden, eine lokale In-Memory Datenbank zu verwenden. Dazu wurde die *H2 Database*[5] gewählt, welche eine nicht-persistente Datenbank bereitstellt. Diese wurden dann mit dem SQL-Schema der Production-Datenbank initialisiert.

Das Mocken von Ktor-Servern wird explizit unterstützt, dafür wurde die *testApplication* genutzt und der *RequestDispatcher* gemockt. Dadurch konnten alle Routen, die der *RequestController* bereitstellt, erfolgreich getestet werden. Die Test-Abdeckung wurde mittels des Intellij-Tools gemessen, es ergibt sich die in Abbildung 2 dargestellte Abdeckung.

[2] https://docs.junit.org/5.10.5/user-guide/
[3] https://mockk.io/
[4] https://github.com/webcompere/system-stubs
[5] https://h2database.com/html/main.html

Current scope: all classes

Overall Coverage Summary

Package	Class, %	Method, %	Branch, %	Line, %
all classes	97,3% (110/113)	87,9% (210/239)	80,4% (291/362)	91,2% (1199/1314)

Coverage Breakdown

Package	Class, %	Method, %	Branch, %	Line, %
main.kotlin.controller	95,8% (46/48)	67,1% (57/85)	75,8% (144/190)	86,9% (546/628)
main.kotlin.controller.exceptions	100% (1/1)	100% (1/1)		100% (2/2)
main.kotlin.data	100% (16/16)	100% (23/23)	100% (8/8)	100% (47/47)
main.kotlin.data.config	100% (3/3)	100% (6/6)	83,3% (5/6)	94,1% (16/17)
main.kotlin.data.events	100% (10/10)	100% (19/19)		100% (48/48)
main.kotlin.database	100% (6/6)	100% (29/29)	100% (2/2)	91,3% (42/46)
main.kotlin.event	94,1% (16/17)	97,7% (43/44)	80,7% (92/114)	94% (328/349)
main.kotlin.user	100% (12/12)	100% (32/32)	95,2% (40/42)	96% (170/177)

generated on 2026-03-09 19:08

Abbildung 2: Testabdeckung des Server-Moduls

Da der RequestDispatcher im Endeffekt nur eine Translation des Request-Controllers auf die Manager darstellt, haben wir uns dazu entschieden, nur die beiden Methoden *createSharedEventAndShare* und *createGlobalChallenge* zu Unit-testen, da diese mehr Funktionaliät bereitstellen als der Rest.

Auch wurden teilweise Exceptions oder abgefangene Fehler nicht erreicht, da Argumente vorher bereits für einen besseren Kontrollfluss und eine aussagekräftigere Fehlermeldung validiert wurden.

5 Integrationstests

Da bereits in der Implementierungsphase Integrationstests für den Server geschrieben wurden, wurden diese in der Qualitätssicherung erweitert, sodass alle Routen getestet werden. Damit wird die Kommunikation zwischen Model und Server getestet, sodass etwaige Fehler wie bspw. das aufgetretene Serialisierungsproblem erkannt werden können. Für den Login wurden dabei JWT Tokens selber generiert[6], da für Testnutzer in Firebase die Tokens zu schnell ablaufen, als das diese eine reproduzierbare Testentwicklung ermöglichen. Die Integrationstests befinden sich im Server unter *IntegrationTests*.

[6] https://www.jwt.io/

6 Systemtests

6.1 Server Verfügbarkeit

Die allegmeine Verfügbarkeit des Servers wurde mittels *uptimerobot.com*[7] über den Zeitraum der Qualitätssicherung, also ca. drei Wochen geprüft. Dazu wurde eine eigene */health* Rest-API Route hinzugefügt, mit dem sich die Erreichbarkeit des Servers einfach testen lässt. Der Server hat eine Erreichbarkeit von 100% erreicht, die durchschnittliche Reponse-Time beträgt ca. 200 ms. Damit sind die technischen Anforderungen an den Server erfüllt worden.

6.2 Manuelle Tests

6.2.1 Login und Logout

Registrierung wurde getestet, indem ein Teammitglied sich mit einem neuen Google-Account angemeldet hat. Es kam die Auswahl für das Google-Konto und danach das Eingabefeld für den Anzeigenamen. Auch auf dem Server war der Account nun gespeichert. Login wurde getestet, indem ein Teammitglied sich mit einem Google-Account angemeldet hat, welcher bereits ein Konto auf dem Server hat. Es kam die Auswahl für das Google-Konto, aber kein Eingabefeld für den Anzeigenamen, da dieser schon existiert. Außerdem meldete sich ein Teammitglied an, ohne sich davor abzumelden, Dabei kam keine Auswahl für das Google-Konto, da das alte noch angemeldet ist. Der Nutzer kam direkt in die App selbst rein, nachdem er den Login-Knopf gedrückt. Logout wurde getestet, indem ein Teammitglied den Profil-Screen aufrief und auf "Abmelden" drückte. Er kam wieder in den Login-Screen, wo er sich erneut anmelden konnte, wobei er ein neues Google-Konto auswählen durfte. Löschung wurde getestet, indem ein Teammitglied in dem Profil-Screen auf "Account löschen" drückte. Er musste die Löschung bestätigen und befand sich danach wieder im Login-Screen, wo er nach der Auswahl des alten Google-Kontos einen Anzeigenamen wählen durfte, da der Account jetzt neu war. Auch auf dem Server war der Account gelöscht, bevor er sich neu angemeldet hatte.

6.2.2 Events

Die Events wurden getestet, indem ein Teammitglied jeweils alle Events erstellte (wobei sie danach sowohl auf dem Server gespeichert, als auch in der App

[7] https://dashboard.uptimerobot.com

angezeigt wurden) bearbeitete (indem er auf sie tippte und die Informationen änderte), abhakte und löschte (wobei sie auch auf dem Server gelöscht wurden). Shared-Events wurden dabei auch jeweils bei dem/den Freund/en angezeigt. Termnine und Shared-Events konnten sowohl mit als auch ohne Gruppe erstellt werden.

6.2.3 Freunde

Ein Teammitglied ist in den Profilscreen gegangen und hat Freunde hinzugefügt und entfernt, sie erschienen und verschwanden sowohl aus der Freundesliste, aus der Liste der Freunde beim Shared-Event, als auch im Server.

6.2.4 Dark-Mode und Sprache

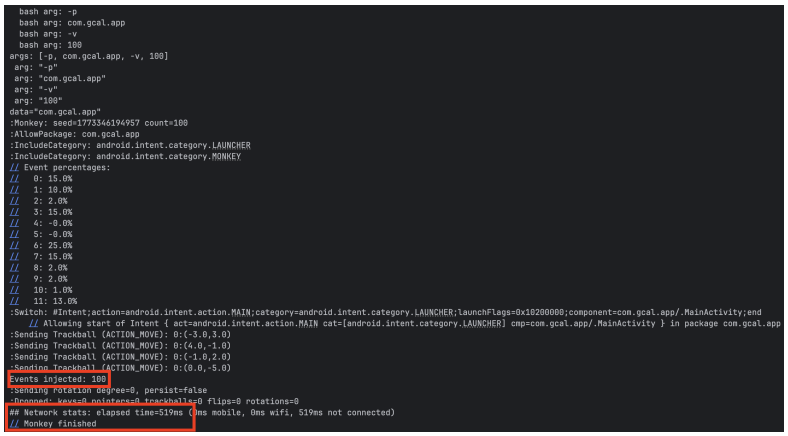
Sie wurden getestet, indem ein Teammitglied auf den Profilscreen wechselte und die Einstellungen aufrief und die Sprache wechselte/den Darkmode aktivierte/deaktivierte. Beides funktionierte so wie erwartet.

6.3 UI-Stresstests (Monkey Testing)

Um die Stabilität und Ausfallsicherheit der Benutzeroberfläche unter extremen Bedingungen zu gewährleisten, wurden automatisierte Stresstests mit dem Android UI/Application Exerciser *Monkey* durchgeführt.

Es wurden drei verschiedene Test-Szenarien durchgeführt:

1. **Basis-Stabilitätstest (Baseline Monkey Test)**
   - **Ziel:** Überprüfung des App-Starts und der initialen Reaktionen der Benutzeroberfläche.
   - **Parameter:** 100 zufällige Interaktionen (Events), ohne Verzögerung.
   - **Ergebnis:** Der Test wurde in ca. 0,5 Sekunden (519 ms) fehlerfrei abgeschlossen.

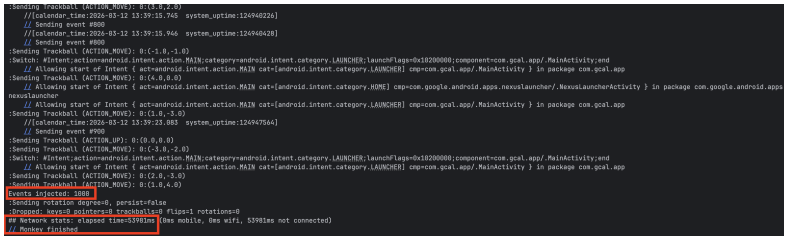


```
  bash arg: -p
  bash arg: com.gcal.app
  bash arg: -v
  bash arg: 100
args: [-p, com.gcal.app, -v, 100]
 arg: "-p"
 arg: "com.gcal.app"
 arg: "-v"
 arg: "100"
data="com.gcal.app"
:Monkey: seed=1773346194957 count=100
:AllowPackage: com.gcal.app
:IncludeCategory: android.intent.category.LAUNCHER
:IncludeCategory: android.intent.category.MONKEY
// Event percentages:
//   0: 15.0%
//   1: 10.0%
//   2: 2.0%
//   3: 15.0%
//   4: -0.0%
//   5: -0.0%
//   6: 25.0%
//   7: 15.0%
//   8: 2.0%
//   9: 2.0%
//   10: 1.0%
//   11: 13.0%
:Switch: #Intent;action=android.intent.action.MAIN;category=android.intent.category.LAUNCHER;launchFlags=0x10200000;component=com.gcal.app/.MainActivity;end
    // Allowing start of Intent { act=android.intent.action.MAIN cat=[android.intent.category.LAUNCHER] cmp=com.gcal.app/.MainActivity } in package com.gcal.app
:Sending Trackball (ACTION_MOVE): 0:(-3.0,3.0)
:Sending Trackball (ACTION_MOVE): 0:(4.0,-1.0)
:Sending Trackball (ACTION_MOVE): 0:(-1.0,2.0)
:Sending Trackball (ACTION_MOVE): 0:(0.0,-5.0)
Events injected: 100
:Sending rotation degree=0, persist=false
:Dropped: keys=0 pointers=0 trackballs=0 flips=0 rotations=0
## Network stats: elapsed time=519ms (0ms mobile, 0ms wifi, 519ms not connected)
// Monkey finished
```

Abbildung 3: Ergebnis des Basis-Stabilitätstests (100 Events)

2. Realistische Benutzersimulation

- **Ziel:** Simulation einer intensiven Nutzung durch einen Anwender.
- **Parameter:** 1000 Events mit 200 ms Delay.
- **Ergebnis:** 54 Sekunden Dauer ohne Crash.



```
:Sending Trackball (ACTION_MOVE): 0:(1.0,2.0)
    //[calendar_time:2026-03-12 13:39:15.745  system_uptime:124946226]
    // Sending event #800
    //[calendar_time:2026-03-12 13:39:15.946  system_uptime:124946428]
    // Sending event #800
:Sending Trackball (ACTION_MOVE): 0:(-1.0,-1.0)
:Switch: #Intent;action=android.intent.action.MAIN;category=android.intent.category.LAUNCHER;launchFlags=0x10200000;component=com.gcal.app/.MainActivity;end
    // Allowing start of Intent { act=android.intent.action.MAIN cat=[android.intent.category.LAUNCHER] cmp=com.gcal.app/.MainActivity } in package com.gcal.app
:Sending Trackball (ACTION_MOVE): 0:(4.0,0.0)
    // Allowing start of Intent { act=android.intent.action.MAIN cat=[android.intent.category.HOME] cmp=com.google.android.apps.nexuslauncher/.NexusLauncherActivity } in package com.google.android.apps.
nexuslauncher
    // Allowing start of Intent { act=android.intent.action.MAIN cat=[android.intent.category.LAUNCHER] cmp=com.gcal.app/.MainActivity } in package com.gcal.app
:Sending Trackball (ACTION_MOVE): 0:(1.0,-3.0)
    //[calendar_time:2026-03-12 13:39:23.083  system_uptime:124947566]
    // Sending event #900
:Sending Trackball (ACTION_UP): 0:(0.0,0.0)
:Sending Trackball (ACTION_MOVE): 0:(-3.0,-2.0)
:Switch: #Intent;action=android.intent.action.MAIN;category=android.intent.category.LAUNCHER;launchFlags=0x10200000;component=com.gcal.app/.MainActivity;end
    // Allowing start of Intent { act=android.intent.action.MAIN cat=[android.intent.category.LAUNCHER] cmp=com.gcal.app/.MainActivity } in package com.gcal.app
:Sending Trackball (ACTION_MOVE): 0:(2.0,-3.0)
:Sending Trackball (ACTION_MOVE): 0:(1.0,4.0)
Events injected: 1000
:Sending rotation degree=0, persist=false
:Dropped: keys=0 pointers=0 trackballs=0 flips=1 rotations=0
## Network stats: elapsed time=53981ms (0ms mobile, 0ms wifi, 53981ms not connected)
// Monkey finished
```

Abbildung 4: Ergebnis der realistischen Benutzersimulation (1000 Events mit 200ms Delay)

3. Maximale Kapazitätsprüfung

- **Ziel:** Test der maximalen Systemlast.
- **Parameter:** 1000 Events ohne Delay.
- **Ergebnis:** Verarbeitung in ca. 6,1 Sekunden ohne Absturz.

```
:Switch: #Intent;action=android.intent.action.MAIN;category=android.intent.category.LAUNCHER;launchFlags=0x10200000;component=com.gcal.app/.MainActivity;end
    // Allowing start of Intent { act=android.intent.action.MAIN cat=[android.intent.category.LAUNCHER] cmp=com.gcal.app/.MainActivity } in package com.gcal.app
:Sending Trackball (ACTION_MOVE): 0:(-3.0,-3.0)
:Sending Trackball (ACTION_MOVE): 0:(3.0,0.0)
    // Rejecting start of Intent { cmp=com.google.android.inputmethod.latin/com.google.android.libraries.inputmethod.permissions.PermissionsActivity } in package com.google.android.inputmethod.latin
:Sending Trackball (ACTION_MOVE): 0:(3.0,-4.0)
    //[calendar_time:2026-03-12 13:55:17.806  system_uptime:125901407]
    // Sending event #800
:Switch: #Intent;action=android.intent.action.MAIN;category=android.intent.category.LAUNCHER;launchFlags=0x10200000;component=com.gcal.app/.MainActivity;end
    // Allowing start of Intent { act=android.intent.action.MAIN cat=[android.intent.category.LAUNCHER] cmp=com.gcal.app/.MainActivity } in package com.gcal.app
:Sending Trackball (ACTION_MOVE): 0:(-1.0,1.0)
:Sending Trackball (ACTION_UP): 0:(0.0,0.0)
:Sending Trackball (ACTION_MOVE): 0:(-1.0,-5.0)
:Sending Trackball (ACTION_UP): 0:(0.0,0.0)
:Sending Trackball (ACTION_MOVE): 0:(3.0,3.0)
:Sending Trackball (ACTION_MOVE): 0:(-2.0,4.0)
:Sending Trackball (ACTION_MOVE): 0:(-2.0,-3.0)
:Sending Flip keyboardOpen=true
Got IOException performing flipjava.io.FileNotFoundException: /dev/input/event0: open failed: EACCES (Permission denied)
    // Injection Failed
    //[calendar_time:2026-03-12 13:55:18.325  system_uptime:125902806]
    // Sending event #900
:Sending Trackball (ACTION_MOVE): 0:(-5.0,0.0)
:Sending Trackball (ACTION_MOVE): 0:(-1.0,0.0)
:Sending Trackball (ACTION_UP): 0:(0.0,0.0)
:Sending Trackball (ACTION_MOVE): 0:(4.0,-1.0)
:Switch: #Intent;action=android.intent.action.MAIN;category=android.intent.category.LAUNCHER;launchFlags=0x10200000;component=com.gcal.app/.MainActivity;end
    // Allowing start of Intent { act=android.intent.action.MAIN cat=[android.intent.category.LAUNCHER] cmp=com.gcal.app/.MainActivity } in package com.gcal.app
:Sending Trackball (ACTION_MOVE): 0:(3.0,4.0)
:Sending Trackball (ACTION_MOVE): 0:(0.0,2.0)
events injected: 1000
:Sending rotation degree=0, persist=false
Dropped: keys=0 pointers=1 trackballs=0 flips=0 rotations=0
** Network stats: elapsed time=6146ms (0ms mobile, 0ms wifi, 6146ms not connected)
// Monkey finished
```

Abbildung 5: Ergebnis der maximalen Kapazitätsprüfung (1000 Events ohne Delay)